DM : Secure Vault

TP d'examen : synthèse de toutes les notions vues depuis le TD05 (NASM, structures, bibliothèque de fonctions `cours.inc`, gestion de la pile, syscalls fichiers). Le sujet complet est dans `cours6/Secure_vault_spec.txt`, le barème dans `cours6/secure_vault_barem.txt`.

+++ EXAM TP !!!

Spécification (Secure_vault_spec.txt) — gestionnaire de mots de passe

- F0 : demande le mot de passe de la base (fichier plat, créé s'il n'existe pas). Ce mot de passe sert de clé pour un chiffrement XOR (ECB, 8 premiers octets de la clé).
- F1 : demande login + password à l'utilisateur (structures de 32 octets chacune), déchiffre la base existante, ajoute la nouvelle entrée, rechiffre l'ensemble, sauvegarde dans le fichier.
- F2 : déchiffre le fichier et affiche son contenu en clair.

Chaîne d'outils (build_nasm.sh)

```
./build_nasm.sh exo1_people_note.asm    # nasm -f elf64 + ld
./exo1_people_note
```

Exercices préparatoires fournis (à faire avant le sujet complet)

- `exo1_people_note.asm` (Exercice 3.1 de cours5/exercices.md) : reprend la structure `people` du TD07 en y ajoutant 3 notes :

```
struc people
    .name: resb 32
    .age: resq 1
    .notes resq 3
endstruc
...
loop_notes:
    cmp r8, 3
    jae end
    call read_hex
    mov [rbx + people.notes + r8*8], rax
    inc r8
    jmp loop_notes
```

Le squelette fourni lit le nom, l'âge et les 3 notes, mais ne calcule ni n'affiche encore la moyenne : c'est le complément attendu de l'étudiant (division par 3

du total des notes, puis affichage via `print_hex` de nom/âge/notes/moyenne).

- `exo2_crack_me.asm` (Exercice 3.2 de cours5/exercices.md) : demande 3 valeurs hexadécimales (`rax`, `rbx`, `rcx`) et vérifie 3 équations linéaires successives (échec -> `fail` si l'une d'elles n'est pas satisfaite) :

```
xor rdx, rdx
add rdx, rbx
shl rax, 1
add rdx, rax
shl rbx, 1
add rdx, rbx
add rdx, rcx
sub rdx, 287
cmp rdx, 0
jne fail
```

Attendu : analyse statique (`objdump -d -M intel` ou Ghidra) pour reconstituer le système de 3 équations à 3 inconnues et trouver le triplet (rax, rbx, rcx) qui satisfait les 3 conditions.

Bibliothèque `inc/cours.inc`

Identique à celle du TD07 (`read_hex`, `print_hex`, `read_string`, `print_string`) — c'est la base à réutiliser/étendre pour le sujet complet (notamment pour lire le mot de passe, les login/password, et gérer le fichier de la base).

Barème (secure_vault_barem.txt)

```
Fonctionnel (10 pts) :
    - état de livraison / compilation ....... 2 + 2 pts
    - F0 (gestion mot de passe / clé) ........ 2 pts
    - F1 (ajout de credentials) ............... 2 pts
    - F2 (affichage déchiffré) ................ 2 pts
Qualité (10 pts) :
    - usage de l'IA raisonable ................ 2 pts
    - usage correct de la pile ................ 4 pts
    - bibliothèque de fonctions réutilisable .... 4 pts
```

Ce qui est attendu de l'étudiant

1. Terminer les deux exercices préparatoires (`exo1_people_note.asm`, `exo2_crack_me.asm`) avant d'attaquer le sujet complet.
2. Étendre `inc/cours.inc` avec les fonctions nécessaires à la gestion de fichiers (`open`/`read`/`write`/`close` via syscalls, cf. Exercice 2.7 de cours4/exercices.md) et au chiffrement XOR ECB (F0/F1/F2).

3. Structurer le code en fonctions réutilisables, avec un usage rigoureux de la pile (prologue/épilogue `push`/`pop`/`enter`/`leave`, sauvegarde des registres) — critère explicitement noté dans le barème.
4. L'usage éventuel d'un assistant IA dans la remise (2 pts du barème), dois etre raisonable. Exemple: pas d'utilisation de notion non abordé dans le cours)